Diese erste Seite bzw. Frontseite ist frei gestaltbar.

Inhaltsverzeichnis

1. verschiedene Zitatmöglichkeiten **5**
1.1. Seitenzahlen . . . 5
1.2. cite - Autor, Jahr . . . 5
1.3. parencite - (Autor, Jahr) . . . 5
1.4. footcite - Fussnote mit Autor, Jahr . . . 5
1.5. footcitetext - Fussnote nicht brauchen . . . 5
1.6. textcite - Autor (Jahr) . . . 5
1.7. smartcite - Fussnote mit Autor, Jahr . . . 5
1.8. cite* - Jahr . . . 5
1.9. parencite* - (Jahr) . . . 5
1.10. supercite - Autor, Jahr . . . 5
1.11. autocite . . . 6
1.12. citeauthor - Autor . . . 6
1.13. citeauthor* - Autor . . . 6
1.14. citetitle - Titel . . . 6

2. Erstellung Dokument inkl. Bibliographie **6**

3. Einleitung **7**
3.1. Ausgangslage und Motivation . . . 7
3.2. Zielsetzung der Arbeit . . . 7
3.3. Aufbau der Arbeit . . . 7

4. Grundlagen Enterprise Architecture **8**
4.1. Definition und Abgrenzung . . . 8
4.2. Enterprise Architecture Frameworks . . . 8
4.2.1. TOGAF - The Open Group Architecture Framework . . . 8
4.2.2. Zachman Framework . . . 9
4.2.3. FEAF - Federal Enterprise Architecture Framework . . . 9
4.2.4. Gartner Enterprise Architecture Framework . . . 10
4.2.5. ArchiMate . . . 10
4.3. Framework-Vergleich . . . 11
4.4. Enterprise Architecture Tools . . . 11
4.4.1. Sparx Systems Enterprise Architect . . . 11
4.4.2. BiZZdesign Enterprise Studio . . . 11
4.4.3. LeanIX . . . 11
4.4.4. Archi . . . 11
4.4.5. ARIS (Software AG) . . . 12
4.5. Domain Driven Design - Kernkonzepte . . . 12
4.5.1. Strategisches Design . . . 12
4.5.2. Taktisches Design . . . 12
4.6. Microservices Architektur . . . 14
4.7. DDD und Microservices: Der perfekte Match . . . 14
4.8. Von DDD zu Microservices: Praktische Umsetzung . . . 14

5. Methodik und Vorgehen **16**

6. Analyse und Ergebnisse **17**
6.1. Ist-Analyse . . . 17
6.2. Soll-Konzept . . . 17
6.3. Bewertung und Empfehlungen . . . 17

7. Diskussion **18**

8. Fazit und Ausblick **19**

A. Anhang **20**

Literatur **21**

Abbildungsverzeichnis

Tabellenverzeichnis

1. Vergleich der EA-Frameworks 11

1. verschiedene Zitatmöglichkeiten

BibLaTex mit *biber* bietet diverse Möglichkeiten ein Zitat zu erfassen. Das Cheatsheet[1] zeigt ein paar Möglichkeiten auf.

1.1. Seitenzahlen

Grundsätzlich kann die Seitenzahlen immer optional in eckigen Klammern mitgegeben werden. Grundsätzlich ist man dort ziemlich frei.

1.2. cite - Autor, Jahr

Mit dem Befehl *cite* wird das Ganze als Text ausgegeben wie Häberli, 2019, S. 4.

1.3. parencite - (Autor, Jahr)

Mit dem Befehl *parencite* mit alles in Klammer (Häberli, 2019, S. 2) erzeugt.

1.4. footcite - Fussnote mit Autor, Jahr

Mit dem Befehl *footcite* wird das Ganze als Fussnote[2].

1.5. footcitetext - Fussnote nicht brauchen

Mit dem Befehl *footcitetext* erzeugt auch eine Fussnote. Jedoch wird die Nummerierung nicht korrekt erzeugt. Deshalb würde ich das nicht einsetzen.

1.6. textcite - Autor (Jahr)

Mit dem Befehl *textcite* wird ein Text gemäss Häberli (2019, S. 56–66) in Klammer gesetzt.

1.7. smartcite - Fussnote mit Autor, Jahr

Mit dem Befehl *smartcite* wird Text[3] wird derselbe Effekt erreicht wie mit *footcite*

1.8. cite* - Jahr

Mit dem Befehl *cite** wird nur das Jahr ohne Autor als Text ausgegeben wie 2003, S. 3.

1.9. parencite* - (Jahr)

Mit dem Befehl *parencite** wird das Jahr in Klammer ausgegeben, ohne vorhergehenden Autor wie (2003, S. 3).

1.10. supercite - Autor, Jahr

Mit dem Befehl *supercite* werden Autor und Jahr ohne Klammer Steinhoff, 2003, S. 3 erzeugt, anlog *cite*.

[1] https://tug.ctan.org/info/biblatex-cheatsheet/biblatex-cheatsheet.pdf
[2] Häberli, 2019, S. 45.
[3] Häberli, 2019, S. 2.

1.11. autocite

Mit *autocite* wird ein Zitat ohne Klammer erstellt wie mit (Steinhoff, 2003, S. 3). Ich könnte mir jedoch vorstellen, dass bei eineer anderen biblatex-Einstellung etwas anderes angezeigt wird. Deshalb eher mit Vorsicht zu geniessen.

1.12. citeauthor - Autor

Mit *citeauthor* wird ein Zitat nur mit dem Autor angegeben, wie es Grimm und Grimm in ihrem Buch dargelegt haben.

1.13. citeauthor* - Autor

Mit *citeauthor** wird ein Zitat nur mit dem Autor angegeben, wie es Grimm und Grimm in ihrem Werk bereits zum Besten gegeben haben. Diese scheint genau dasselbe zu sein wie oben.

1.14. citetitle - Titel

Mit *citetitle* wird bei einem Zitat auf den Titel Bezug genommen, wie es im Buch *Kinder- und Hausmärchen*, S. 3 erwähnt wird.

2. Erstellung Dokument inkl. Bibliographie

Mit dem Befehl *pdflatex dateiname* wird das Ganze normal erstellt und mit *biber dateiname* wird für die Bibliographie benötigt. *dateiname* sollte ohne Endung eingegeben werden, weil *biber* damit ein Problem hat.

Lustiges

TEXist eine schöne Darstellung wie LATEX. Ist dasselbe auch mit BibTex möglich?

3. Einleitung

3.1. Ausgangslage und Motivation

tbd

3.2. Zielsetzung der Arbeit

tbd

3.3. Aufbau der Arbeit

tbd

4. Grundlagen Enterprise Architecture

4.1. Definition und Abgrenzung

Enterprise Architecture (EA) ist ein ganzheitlicher Ansatz zur Gestaltung und Steuerung der Unternehmensarchitektur. Sie umfasst die strategische Ausrichtung von Business, Daten, Applikationen und Technologie-Infrastruktur.

EA schafft Transparenz über die IT-Landschaft, unterstützt strategische Entscheidungen und ermöglicht eine effiziente Transformation der Organisation. Ein gut definiertes Enterprise Architecture Framework hilft dabei, die Komplexität zu reduzieren, Redundanzen zu vermeiden und die Ausrichtung zwischen Business und IT sicherzustellen.

4.2. Enterprise Architecture Frameworks

EA-Frameworks bieten strukturierte Methoden und Best Practices für die Entwicklung und Verwaltung von Unternehmensarchitekturen. Im Folgenden werden fünf wesentliche Frameworks detailliert vorgestellt:

4.2.1. TOGAF - The Open Group Architecture Framework

TOGAF ist eines der am weitesten verbreiteten und ausgereiftesten EA-Frameworks weltweit. Das Framework wurde von The Open Group entwickelt und bietet eine umfassende Methodik für die Entwicklung, Implementierung und Wartung von Enterprise Architectures.

Architecture Development Method (ADM) Das Herzstück von TOGAF ist die Architecture Development Method, ein iterativer Prozess mit acht Phasen:

- **Preliminary Phase:** Framework und Prinzipien etablieren
- **Phase A - Architecture Vision:** Scope und Stakeholder-Anforderungen definieren
- **Phase B - Business Architecture:** Geschäftsprozesse und Organisationsstruktur modellieren
- **Phase C - Information Systems Architecture:** Daten- und Applikationsarchitektur entwickeln
- **Phase D - Technology Architecture:** IT-Infrastruktur und Technologie-Plattformen definieren
- **Phase E - Opportunities & Solutions:** Implementierungsplanung und Roadmap erstellen
- **Phase F - Migration Planning:** Detaillierte Migration und Transformation planen
- **Phase G - Implementation Governance:** Architektur-Compliance sicherstellen
- **Phase H - Architecture Change Management:** Kontinuierliche Verbesserung und Änderungsmanagement

Kernkomponenten

- **Enterprise Continuum:** Klassifizierung von Architekturen von generisch bis spezifisch
- **Architecture Repository:** Zentrale Ablage für alle Architektur-Artefakte
- **Architecture Governance:** Prozesse zur Sicherstellung der Architektur-Compliance
- **Architecture Content Framework:** Strukturierte Beschreibung von Architektur-Deliverables

Stärken

- Umfassende, erprobte Methodik mit weltweiter Verbreitung
- Starke Community und etablierte Zertifizierungsprogramme
- Sehr gut dokumentiert und standardisiert
- Eignet sich besonders für große, komplexe Organisationen und Transformationsprojekte

4.2.2. Zachman Framework

Das Zachman Framework ist ein taxonomisches Schema zur Klassifizierung und Organisation von Enterprise Architecture Artefakten. Es wurde von John Zachman entwickelt und basiert auf einer systematischen 6x6-Matrix.

Die 6x6 Matrix Das Framework betrachtet Architekturen aus zwei Dimensionen:

Perspektiven (Zeilen):

- **Planner:** Scope und Kontext (Stakeholder-Perspektive)
- **Owner:** Business-Modell (Executive-Perspektive)
- **Designer:** System-Modell (Architekten-Perspektive)
- **Builder:** Technologie-Modell (Entwickler-Perspektive)
- **Subcontractor:** Detaillierte Spezifikationen (Implementierer-Perspektive)
- **User:** Operatives System (Nutzer-Perspektive)

Fragestellungen (Spalten):

- **What:** Daten und Entities
- **How:** Funktionen und Prozesse
- **Where:** Netzwerk und Standorte
- **Who:** Menschen und Organisationen
- **When:** Zeit und Ereignisse
- **Why:** Motivation und Ziele

Stärken

- Klare Strukturierung und umfassende Taxonomie
- Perspektivenvielfalt ermöglicht ganzheitliche Betrachtung
- Framework-agnostisch und kombinierbar mit anderen Ansätzen
- Fokus auf verschiedene Stakeholder-Sichten
- Ideal für Taxonomie und systematisches Artefakt-Management

4.2.3. FEAF - Federal Enterprise Architecture Framework

FEAF wurde ursprünglich für US-Bundesbehörden entwickelt und ist besonders in stark regulierten Umgebungen verbreitet. Das Framework fokussiert auf Governance, Compliance und standardisierte Architekturbeschreibungen.

Vier Architektur-Domänen

- **Business Architecture:** Geschäftsprozesse, Organisationsstrukturen, Geschäftsfunktionen
- **Data Architecture:** Datenmodelle, Datenflüsse, Informationsmanagement
- **Application Architecture:** Applikationslandschaft, Schnittstellen, Funktionen
- **Technology Architecture:** IT-Infrastruktur, Plattformen, Standards

Stärken

- Starker Fokus auf Governance, Compliance und regulatorische Anforderungen
- Bewährt im öffentlichen Sektor und regulierten Industrien
- Detaillierte Referenzmodelle und standardisierte Dokumentation
- Eignet sich besonders für Behörden und compliance-kritische Organisationen

4.2.4. Gartner Enterprise Architecture Framework

Das Gartner EA Framework ist ein praxisorientierter Ansatz, der stark auf Business-Value und pragmatische Umsetzung fokussiert. Es berücksichtigt aktuelle Trends wie Digitalisierung, Cloud und Agilität.

Business-getriebener Ansatz Gartner betont die Ausrichtung der EA auf Geschäftswert und messbare Ergebnisse:

- **Value Delivery:** Fokus auf Business-Outcomes statt rein technischer Artefakte
- **Continuous Improvement:** Iterative, agile Entwicklung der Architektur
- **Pragmatic Implementation:** Praktische Umsetzbarkeit vor theoretischer Vollständigkeit

Stärken

- Praxisnah und geschäftsorientiert
- Berücksichtigung aktueller Technologie-Trends und Best Practices
- Regelmäßige Updates durch umfangreiche Gartner-Research
- Eignet sich für agile Organisationen mit Fokus auf schnelle Ergebnisse

4.2.5. ArchiMate

ArchiMate ist eine standardisierte Modellierungssprache für Enterprise Architecture, entwickelt von The Open Group. Im Gegensatz zu den anderen Frameworks ist ArchiMate primär eine visuelle Notationssprache.

Drei Architektur-Layer

- **Business Layer:** Geschäftsprozesse, Akteure, Services, Produkte
- **Application Layer:** Applikationen, Komponenten, Schnittstellen, Daten
- **Technology Layer:** Infrastruktur, Netzwerke, Plattformen, Systeme

Beziehungstypen ArchiMate definiert verschiedene Beziehungstypen zwischen Elementen:

- Strukturelle Beziehungen (Composition, Aggregation, Assignment)
- Abhängigkeitsbeziehungen (Serving, Access, Influence)
- Dynamische Beziehungen (Flow, Triggering)
- Andere (Specialization, Realization, Association)

Stärken

- Standardisierte Notation (Open Group Standard)
- Hervorragende Visualisierungsmöglichkeiten für komplexe Zusammenhänge
- Breite Tool-Unterstützung durch verschiedene EA-Tools
- Sehr gut integrierbar mit TOGAF
- Ideal für visuelle Kommunikation von Architektur-Entscheidungen

4.3. Framework-Vergleich

Die verschiedenen EA-Frameworks haben unterschiedliche Stärken und eignen sich für unterschiedliche Anwendungsfälle. Tabelle 1 gibt einen Überblick über die Hauptcharakteristika.

Tabelle 1: Vergleich der EA-Frameworks

Framework	Hauptstärke	Typischer Anwendungsfall
TOGAF	Umfassende Methodik	Große Organisationen, komplexe Transformationen
Zachman	Strukturierte Klassifikation	Taxonomie, Artefakt-Management
FEAF	Compliance-fokussiert	Behörden, regulierte Industrien
Gartner	Business-Value, pragmatisch	Agile Organisationen, schnelle Ergebnisse
ArchiMate	Visuelle Kommunikation	Modellierung, Stakeholder-Kommunikation

4.4. Enterprise Architecture Tools

Für die praktische Umsetzung von EA stehen verschiedene spezialisierte Werkzeuge zur Verfügung. Die Wahl des richtigen Tools sollte Framework-Unterstützung, Kollaborationsmöglichkeiten und Integration in die bestehende Tool-Landschaft berücksichtigen.

4.4.1. Sparx Systems Enterprise Architect

Umfassendes Modellierungs-Tool mit Unterstützung für UML, BPMN, SysML und ArchiMate. Enterprise Architect ist besonders stark in technischer Dokumentation, Code-Engineering und der Integration verschiedener Modellierungsstandards.

4.4.2. BiZZdesign Enterprise Studio

ArchiMate-native, kollaborative EA-Plattform mit starkem Fokus auf Business-IT-Alignment und strategische Planung. BiZZdesign bietet umfangreiche Analysemöglichkeiten und Dashboards für Entscheidungsträger.

4.4.3. LeanIX

Moderne, cloud-basierte SaaS-Lösung mit intuitivem Interface. Fokus auf Application Portfolio Management, Cloud-Transformation und IT-Landschaftsoptimierung. LeanIX zeichnet sich durch Benutzerfreundlichkeit und schnelle Time-to-Value aus.

4.4.4. Archi

Open-Source-Tool für ArchiMate-Modellierung. Kostenfrei, leichtgewichtig und ideal für Einstieg in EA-Modellierung sowie kleinere Teams. Trotz des Open-Source-Charakters bietet Archi eine vollständige ArchiMate-Unterstützung.

4.4.5. ARIS (Software AG)

Enterprise-Suite mit starkem Fokus auf Business Process Management, Governance, Risk & Compliance (GRC). ARIS integriert Prozessmanagement mit Enterprise Architecture und bietet umfangreiche Analyse- und Simulationsmöglichkeiten.

4.5. Domain Driven Design - Kernkonzepte

Domain Driven Design (DDD) ist ein Ansatz zur Softwareentwicklung für komplexe Domänen. DDD fokussiert auf die Geschäftslogik und die gemeinsame Sprache zwischen Entwicklern und Domänen-Experten. Die Kernidee: Software spiegelt die Geschäftsdomäne wider und entwickelt sich gemeinsam mit dem Verständnis der Domäne.

4.5.1. Strategisches Design

Das strategische Design in DDD befasst sich mit der großflächigen Strukturierung der Domäne:

Bounded Context Ein Bounded Context definiert eine explizite Grenze, innerhalb derer ein bestimmtes Domänenmodell gültig ist. Jeder Bounded Context hat seine eigene Ubiquitous Language.

Warum Bounded Contexts?

- Ein einziges Modell für alles ist unmöglich und unpraktisch
- Begriffe haben in verschiedenen Kontexten unterschiedliche Bedeutungen
- Verschiedene Perspektiven erfordern unterschiedliche Modelle
- Reduziert kognitive Last durch klare Abgrenzungen

Beispiel "Kunde":

- **Sales Context:** Kunde = Leads, Opportunities, Verkaufschancen
- **Shipping Context:** Kunde = Lieferadresse, Tracking-Informationen
- **Support Context:** Kunde = Tickets, SLA-Vereinbarungen

Context Mapping Context Mapping beschreibt die Beziehungen zwischen verschiedenen Bounded Contexts:

- **Shared Kernel:** Gemeinsamer Code zwischen Contexts
- **Customer/Supplier:** Abhängigkeitsbeziehung zwischen Teams
- **Anti-Corruption Layer:** Schutzschicht gegen externe Modelle
- **Conformist:** Übernahme des externen Modells
- **Separate Ways:** Keine Integration notwendig

Ubiquitous Language Eine gemeinsame, allgegenwärtige Sprache zwischen Entwicklern und Domänen-Experten. Diese Sprache:

- Wird im Code, in Dokumentation und in Gesprächen verwendet
- Reduziert Missverständnisse und Übersetzungsfehler
- Ist context-spezifisch (kann zwischen Bounded Contexts variieren)
- Entwickelt sich kontinuierlich mit dem Domänenverständnis

4.5.2. Taktisches Design

Das taktische Design fokussiert auf die Implementierung innerhalb eines Bounded Context:

Entities Objekte mit eindeutiger Identität über ihren gesamten Lebenszyklus hinweg. Die Identität bleibt erhalten, auch wenn sich Attribute ändern.

Beispiel: Ein Kunde mit ID 12345 bleibt derselbe Kunde, auch wenn Name oder Adresse geändert werden.

Value Objects Immutable Objekte ohne konzeptionelle Identität, vollständig definiert durch ihre Attribute.

Beispiel: Eine Adresse oder ein Geldbetrag. Zwei Adressen mit identischen Werten sind austauschbar.

Vorteile:

- Einfacher zu testen und zu verstehen
- Thread-safe durch Immutability
- Keine Identitätsverwaltung notwendig

Aggregates Ein Cluster von Objekten, die als Einheit behandelt werden, mit einem Aggregate Root als einzigem Einstiegspunkt.

Regeln für Aggregates:

- Zugriff auf Aggregate-Interna nur über die Aggregate Root
- Die Root ist verantwortlich für die Wahrung aller Invarianten
- Transaktionsgrenzen entsprechen Aggregate-Grenzen
- Referenzen zwischen Aggregates nur über IDs, nicht über direkte Objektreferenzen

Beispiel Order Aggregate:

- **Root:** Order (Bestellung)
- **Entities:** OrderLine (Bestellposition)
- **Value Objects:** ShippingAddress, Money
- **Invariante:** Gesamtsumme = Summe aller Bestellpositionen

Design-Tipp: Small Aggregates bevorzugen. Nur zusammengehörige Daten in ein Aggregate packen. Eventual Consistency zwischen Aggregates akzeptieren.

Domain Events Ereignisse, die etwas Signifikantes in der Domäne ausdrücken (in der Vergangenheitsform).

Beispiele:

- OrderPlaced (Bestellung aufgegeben)
- PaymentReceived (Zahlung eingegangen)
- CustomerRelocated (Kunde umgezogen)

Vorteile:

- Entkopplung zwischen Bounded Contexts
- Audit-Log und Event Sourcing möglich
- Basis für Event-Driven Architecture

Repositories und Services **Repositories:** Zugriff auf Aggregates, Abstraktion der Persistenz

Domain Services: Operationen, die nicht natürlich zu einer Entity oder einem Value Object gehören

4.6. Microservices Architektur

Microservices ist ein Architekturstil, bei dem eine Anwendung aus kleinen, unabhängigen Services besteht.

Kernprinzipien

- **Single Responsibility:** Jeder Service hat eine klare, fokussierte Aufgabe
- **Autonomy:** Services können unabhängig entwickelt, deployed und skaliert werden
- **Decentralization:** Dezentrale Datenhaltung und Governance
- **Resilience:** Fehler in einem Service beeinträchtigen nicht das gesamte System
- **Observable:** Logging, Monitoring und Tracing sind essenziell

Charakteristika

- Losgelöste Deployments
- Polyglot Persistence (verschiedene Datenbanken pro Service)
- API-basierte Kommunikation (REST, gRPC, Events)
- Organisiert um Business Capabilities

4.7. DDD und Microservices: Der perfekte Match

Domain Driven Design und Microservices ergänzen sich ideal:

Warum DDD und Microservices zusammenpassen

- **Bounded Context = Service Boundary:** DDD definiert natürliche Servicegrenzen
- **Ubiquitous Language:** Klare Kommunikation innerhalb und zwischen Services
- **Aggregates:** Definieren Transaktionsgrenzen und Konsistenz-Boundaries
- **Strategic Patterns:** Context Mapping hilft bei Service-Integration
- **Business Alignment:** Beide Ansätze fokussieren auf Business-Value

Bounded Contexts als Microservices Jeder Bounded Context kann als eigenständiger Microservice implementiert werden:

- Klare Grenzen und Verantwortlichkeiten
- Autonome Teams können an verschiedenen Contexts arbeiten
- Unabhängige Deployments pro Context möglich
- Technologie-Stack kann pro Context variieren

4.8. Von DDD zu Microservices: Praktische Umsetzung

Schritt 1: Event Storming Workshop Gemeinsam mit Domänen-Experten die Geschäftsprozesse und Events identifizieren. Events in der Vergangenheitsform formulieren (OrderPlaced, PaymentReceived).

Schritt 2: Bounded Contexts identifizieren Zusammengehörige Events und Kommandos zu Contexts gruppieren. Sprachliche Hinweise beachten (gleiche Begriffe mit unterschiedlicher Bedeutung = verschiedene Contexts).

Schritt 3: Aggregates & Service Boundaries definieren Aggregates innerhalb der Bounded Contexts identifizieren. Aggregates als Basis für API-Design nutzen.

Schritt 4: Microservice pro Bounded Context implementieren 1:1 Mapping zwischen Bounded Context und Microservice als Ausgangspunkt. Bei Bedarf können große Contexts später weiter aufgeteilt werden.

Schritt 5: API-Design & Integration Patterns

- **Synchrone Kommunikation:** REST oder gRPC für Abfragen
- **Asynchrone Kommunikation:** Domain Events für lose Kopplung
- **Anti-Corruption Layer:** Schutz vor externen Änderungen

Grundprinzip Start with the domain, let technology follow. Die Architektur sollte die Geschäftsdomäne widerspiegeln, nicht umgekehrt.

5. Methodik und Vorgehen

tbd

6. Analyse und Ergebnisse

6.1. Ist-Analyse

tbd

6.2. Soll-Konzept

tbd

6.3. Bewertung und Empfehlungen

tbd

7. Diskussion

tbd

8. Fazit und Ausblick

tbd

A. Anhang

Literatur

Grimm, W., & Grimm, J. (1812). *Kinder- und Hausmärchen*. Georg Andreas Reimer.
Häberli, S. (2019). Ernüchternde Öko-Bilanz: Photovoltaik und Biogas schaden dem Klima. *NZZ*, 1.
Steinhoff, T. (2003). Wie entwickelt sich die wissenschaftliche Textkompetenz? *Der Deutschunterricht*, (3), 38–47.